IQRA University		
Faculty of Engineering Sciences and Technology (FEST)		
Department of Computer Science		
Course Code	**Course Name**	**Credit Hr.**
CSC 223-L	Computer Architecture Lab	2+1

Lab 1 &2

Week No 1.

Lab Description:

1 Exploring Instruction Set Architecture (ISA) of x86 Machines

2 Learning to program in Assembly Language of x86 Machines

Install the EMU8086 emulator, which supports 16-bit registers and provides an easy-to-use interface for learning Assembly programming. This emulator helps students visualize how low-level instructions are executed inside the CPU. By running and experimenting with the given sample codes, students will gain a practical understanding of how Assembly language interacts with the processor, memory, and registers.

Through these exercises, students will also learn key concepts such as the **data segment, code segment, interrupts, and DOS functions**, which form the foundation of **computer architecture and low-level system programming**.

In an 8086 program, memory is divided into segments — like folders in a computer. Each segment stores a specific kind of thing.

Segment	Purpose
CODE segment	Stores the program instructions (what CPU will execute).
DATA segment	Stores variables, constants, and strings (the information your code uses).
STACK segment	Stores temporary data for function calls, interrupts, etc.

Typical EMU8086 (MASM-style) program looks like this:

```
.MODEL SMALL
.STACK 100H

.DATA
; variables here

.CODE
MAIN PROC
    ; instructions here
MAIN ENDP
```

```
END MAIN
```

.MODEL SMALL

This tells the assembler **how big the program will be** and how memory will be divided

There are different models:

Model	Meaning
TINY	One segment (both code and data together)
SMALL	One CODE and one DATA segment ✅ *(most common for students)*
MEDIUM	Multiple code segments, one data segment
LARGE	Multiple code and data segments

```
+-----------------------+
|   STACK SEGMENT       |  ← temporary memory (100H bytes)
+-----------------------+
|   DATA SEGMENT (DS)   |  ← variables, messages
|   msg DB 'HELLO$'     |
|   num1 DB 10H          |
+-----------------------+
|   CODE SEGMENT (CS)   |  ← program instructions
|   MOV AX, @DATA        |
|   MOV DS, AX           |
|   LEA DX, msg          |
|   MOV AH, 09H          |
|   INT 21H              |
+-----------------------+
```

1. Displaying a Message

```
; Program 1: Display "HELLO"
.MODEL SMALL
.STACK 100H

.DATA
msg DB 'HELLO$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, msg       ; Load address of message
    MOV AH, 09H       ; DOS print function
    INT 21H           ; Display string

    MOV AH, 4CH       ; Exit to DOS
    INT 21H
MAIN ENDP
END MAIN
```

 Concepts: `.DATA`, `.CODE`, string display using DOS interrupt `INT 21H`.

In Assembly, you can ask DOS to:

Function	What it does	Code
Display text on screen	Show a message	`MOV AH, 09H` + `INT 21H`
Read a key from keyboard	Take input from user	`MOV AH, 01H` + `INT 21H`
Print one character	Show one letter	`MOV AH, 02H` + `INT 21H`
End the program	Exit to DOS prompt	`MOV AH, 4CH` + `INT 21H`

➕ 2. Add Two 8-bit Numbers

```
; Program 2: Add Two 8-bit Numbers
.MODEL SMALL
.STACK 100H

.DATA
num1 DB 10H
num2 DB 20H
result DB ?

.CODE
```

```
MAIN PROC
    MOV AL, num1
    ADD AL, num2
    MOV result, AL

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

 Concepts: `DB` data type, `MOV`, `ADD`, register operations.

➖ 3. Subtract Two 8-bit Numbers

```
; Program 3: Subtract Two 8-bit Numbers
.MODEL SMALL
.STACK 100H

.DATA
num1 DB 50H
num2 DB 30H
result DB ?

.CODE
MAIN PROC
    MOV AL, num1
    SUB AL, num2
    MOV result, AL

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

 Concepts: `SUB` instruction, data transfer, storing results in memory.

---

4. Add Two 16-bit Numbers

```
; Program 4: Add Two 16-bit Numbers
.MODEL SMALL
.STACK 100H

.DATA
num1 DW 1234H
num2 DW 1111H
sum DW ?

.CODE
```

```
MAIN PROC
    MOV AX, num1
    ADD AX, num2
    MOV sum, AX

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

 Concepts: `DW` for 16-bit data, `AX` register, arithmetic on word data.

□ 5. Display Register Content (AL Register)

```
; Program 5: Display Register Content (AL)
.MODEL SMALL
.STACK 100H

.DATA
msg DB 'Result: $'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AL, 41H       ; ASCII of 'A'

    LEA DX, msg
    MOV AH, 09H
    INT 21H            ; Display message

    MOV DL, AL         ; Move value of AL to DL for display
    MOV AH, 02H
    INT 21H            ; Display character in AL

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

 Concepts: Register display using `INT 21H`, ASCII characters.

💻 6. Take User Input and Display It

```
; Program 6: Take One Character Input and Display It
.MODEL SMALL
.STACK 100H

.DATA
msg1 DB 'Enter a character: $'
msg2 DB 0DH,0AH,'You entered: $'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, msg1
    MOV AH, 09H
    INT 21H

    MOV AH, 01H        ; Input character
    INT 21H
    MOV BL, AL         ; Store input in BL

    LEA DX, msg2
    MOV AH, 09H
    INT 21H

    MOV DL, BL
    MOV AH, 02H
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

 Concepts: Input/output using interrupts `01H`, `02H`, and `09H`